CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir

Habib University, Fall 2025

Inheritance

Runtime Polymorphism

We'll start with a concept that will set the stage for polymorphism.

https://www.learncpp.com/cpp-tutorial/pointers-and-references-to-the-base-class-of-derived-objects/

Note the methods. **getName()** is redefined in **Derived**.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

Note

*This is **not** a case of function overloading, as specified in the last slide deck. In fact if you notice, the rules of overloading aren't even being followed here – both functions don't have unique signatures.*

This is a case of function hiding. Therefore, the signature and the return type can be whatever (while keeping the same function name). You could have had a different return type and no **const** *as well,*

void Derived::getName();

Having the same name causes **Base::getName()** *to be hidden by* **Derived::getName()**.

Btw, **const** *is part of function signatures.*

The **main()** function seems fine.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

Accessing the object's attributes directly.

Through a reference.

Through a pointer.

```
int main()
{
    Derived derived{ 5 };
    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    Derived& rDerived{ derived };
    std::cout << "rDerived is a "
              << rDerived.getName()
              << " and has value "
              << rDerived.getValue() << '\n';

    Derived* pDerived{ &derived };
    std::cout << "pDerived is a "
              << pDerived->getName()
              << " and has value "
              << pDerived->getValue() << '\n';

    return 0;
}
```

This produces the following output:

```
derived is a Derived and has value 5
rDerived is a Derived and has value 5
pDerived is a Derived and has value 5
```

Now this is a bit weird; regardless, C++ allows this.

```cpp
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```cpp
int main()
{
    Derived derived{ 5 };

    // These are both legal!
    Base& rBase{ derived };
    Base* pBase{ &derived };

    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    std::cout << "rBase is a "
              << rBase.getName()
              << " and has value "
              << rBase.getValue() << '\n';

    std::cout << "pBase is a "
              << pBase->getName()
              << " and has value "
              << pBase->getValue() << '\n';

    return 0;
}
```

Through a `Base` reference.

Through a `Base` pointer.

Recall that child objects are composed of multiple parts.

It has one part for each inherited class, and a part for itself.

So a pointer or a reference of a parent type can be used to point or refer to a child type object.

However, there is a catch with this.
Because **rBase** and **pBase** are of **Base** type, they can only see members of **Base** (or any classes that **Base** inherited).
But they cannot see members in **Derived**.

```
int main()
{
    Derived derived{ 5 };

    // These are both legal!
    Base& rBase{ derived };
    Base* pBase{ &derived };

    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    std::cout << "rBase is a "
              << rBase.getName()
              << " and has value "
              << rBase.getValue() << '\n';

    std::cout << "pBase is a "
              << pBase->getName()
              << " and has value "
              << pBase->getValue() << '\n';

    return 0;
}
```

Through a `Base` reference.

Through a `Base` pointer.

Note the outputs.

rBase and **pBase** are a **Base** reference and pointer, they can only see members of **Base** (or any classes that **Base** inherited).

```
int main()
{
    Derived derived{ 5 };

    // These are both legal!
    Base& rBase{ derived };
    Base* pBase{ &derived };

    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    std::cout << "rBase is a "
              << rBase.getName()
              << " and has value "
              << rBase.getValue() << '\n';

    std::cout << "pBase is a "
              << pBase->getName()
              << " and has value "
              << pBase->getValue() << '\n';

    return 0;
}
```

This produces the result:

```
derived is a Derived and has value 5
rBase is a Base and has value 5
pBase is a Base and has value 5
```

Through a `Base` reference.

Through a `Base` pointer.

Could you do **pBase->getValueDoubled();**

Nope, **rBase** *and* **pBase** *can't see members in* **Derived**.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

Through a `Base` *reference.*

Through a `Base` *pointer.*

```
int main()
{
    Derived derived{ 5 };

    // These are both legal!
    Base& rBase{ derived };
    Base* pBase{ &derived };

    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    std::cout << "rBase is a "
              << rBase.getName()
              << " and has value "
              << rBase.getValue() << '\n';

    std::cout << "pBase is a "
              << pBase->getName()
              << " and has value "
              << pBase->getValue() << '\n';

    return 0;
}
```

Could you create an array that has **Base** and **Derived** objects?

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
int main()
{
    Derived derived{ 5 };

    // These are both legal!
    Base& rBase{ derived };
    Base* pBase{ &derived };

    std::cout << "derived is a "
              << derived.getName()
              << " and has value "
              << derived.getValue() << '\n';

    std::cout << "rBase is a "
              << rBase.getName()
              << " and has value "
              << rBase.getValue() << '\n';

    std::cout << "pBase is a "
              << pBase->getName()
              << " and has value "
              << pBase->getValue() << '\n';

    return 0;
}
```

Through a `Base` reference.

Through a `Base` pointer.

We need a data type that can manage both types of objects – **Base** and **Derived** in our example.

We saw that parent type pointers can point to both parent objects (obviously), and also child objects with the limitation that the object's own members (in the child class) can't be accessed through the parent type pointer (based on what we've seen thus far).

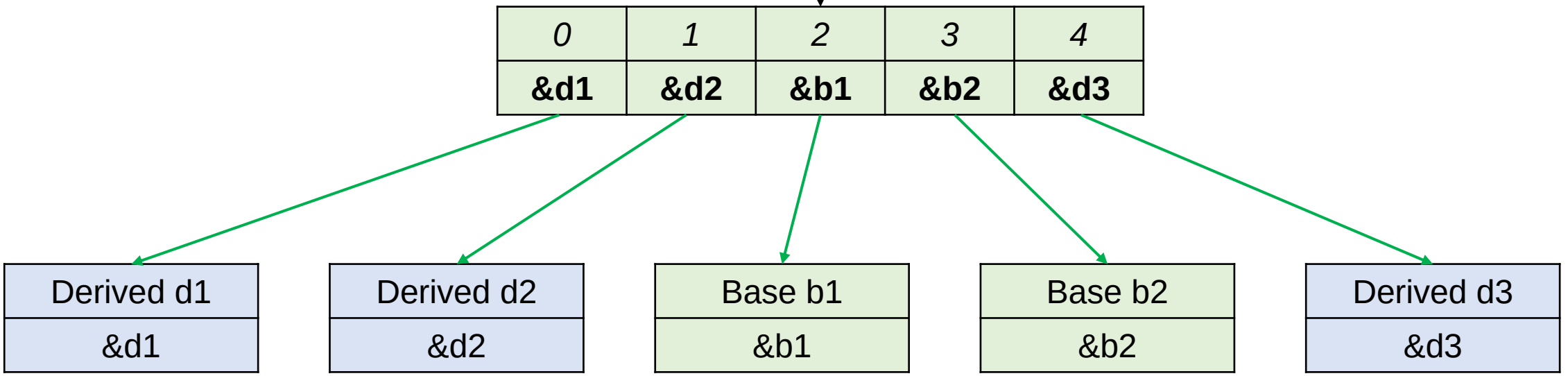
Therefore, we can do the following.

```
int main()
{
    Derived d1{ 10 };
    Derived d2{ 20 };
    Base b1{ 100 };
    Base b2{ 200 };
    Derived d3{ 30 };

    Base* bPtrs[]{ &d1, &d2, &b1, &b2, &d3 };

    return 0;
}
```

```
int main()
{
    Derived d1{ 10 };
    Derived d2{ 20 };
    Base b1{ 100 };
    Base b2{ 200 };
    Derived d3{ 30 };

    Base* bPtrs[]{ &d1, &d2, &b1, &b2, &d3 };

    return 0;
}
```

This is a big benefit of having parent type pointers being able to point at child objects.

Do note, however, that through these **Base***, *we have sacrificed the accessibility of* **Derived** *type objects.*

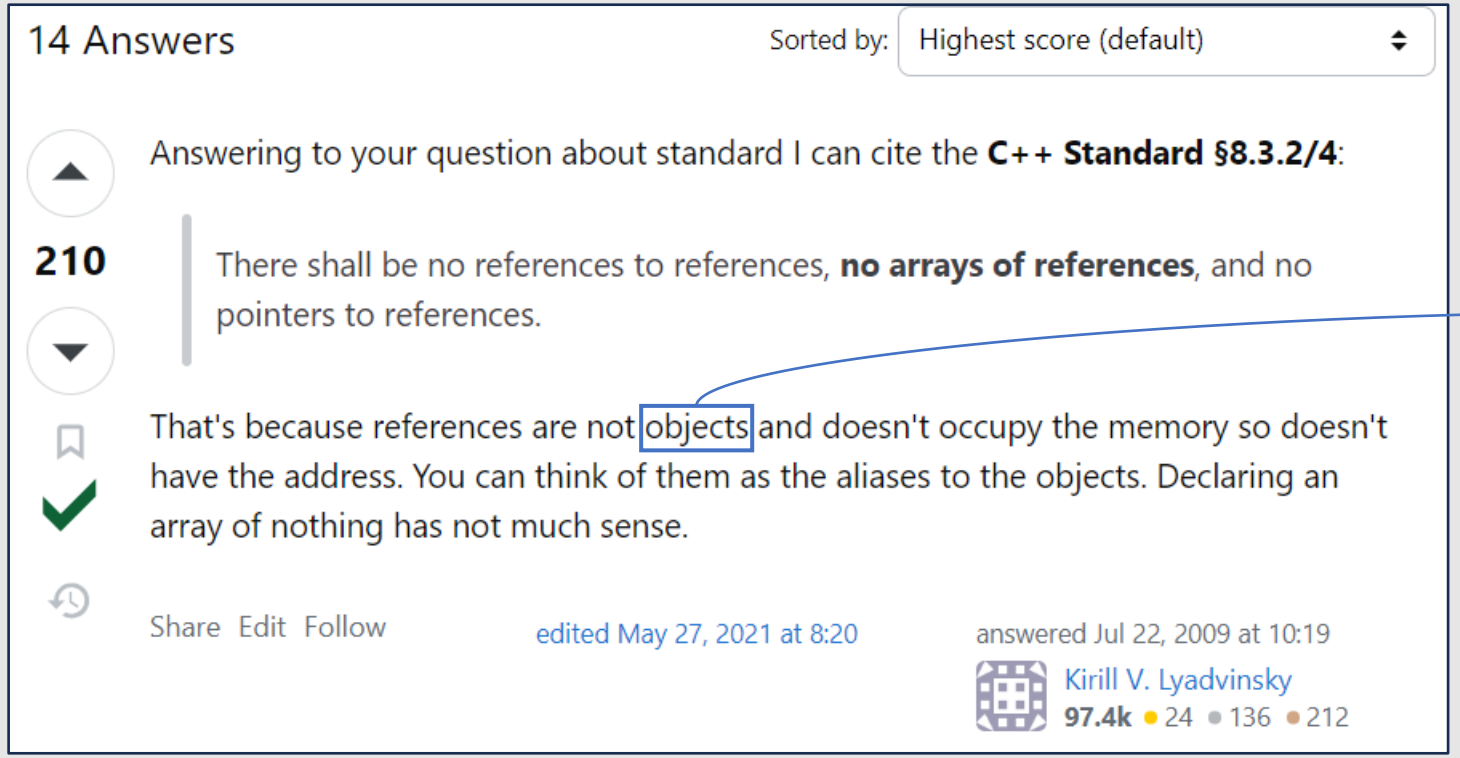


(Optional) You cannot create an array of references. Therefore we would need an array of **Base*** so that we can hold pointers to **Base** and **Derived** objects (in the last example).

If you do want to work with arrays of references, here are some links to help you out.

https://www.learncpp.com/cpp-tutorial/arrays-of-references-via-stdreference_wrapper/

https://stackoverflow.com/questions/1164266/why-are-arrays-of-references-illegal



*Think of objects as **any** entity or variable in this context.*

Another big benefit of this capability is that we can avoid writing overloaded functions for each derived class.

Let's say I have the following function (**not a member of any class**, but is a normal function in the global scope). The **Base** and **Derived** classes are defined as shown earlier.

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}

int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    b1.whoAmI();

    return 0;
}
```

(Ungraded) Quiz

What is the output of this program?

1. *Base, 100*
2. *Base, 10*
3. *Derived, 100*
4. *Derived, 10*
5. *Compiler error*

This would be the output.

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}

int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(b1);

    return 0;
}
```

```
I am Base with value = 100
```

This will work with **d1** as well because parent references can be aliases for child objects.

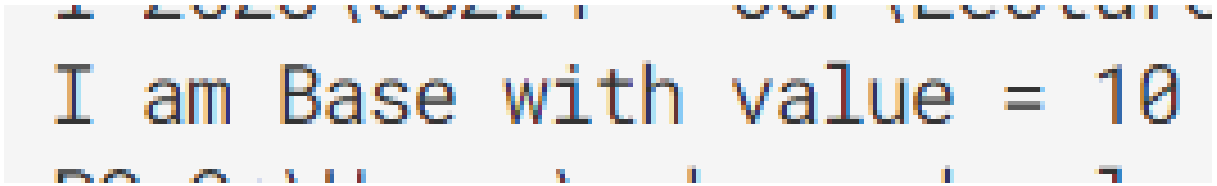
The problem is the same here that the **Base&** cannot see **Derived**'s own members, therefore, it is utilizing **Base**'s implementation of **getName()**.

In case of **getValue()**, *again* **Base***'s implementation is being used, but that is how we want it to be.*

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}

int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    // whoAmI(b1);
    whoAmI(d1);

    return 0;
}
```

As mentioned earlier, the benefit here is that you don't need to write separate overloads for the two classes.

In the future, if there are more derived classes, this single function would suffice.



Virtual Functions

These help in achieving (runtime) polymorphism.

https://www.learncpp.com/cpp-tutorial/virtual-functions/

Virtual Functions

- The sacrifice we made: *The parent type pointer cannot see the child's members.*

- Marking our member functions **virtual** will change this behavior.

Because **obj** is a **Base&**, **obj.getName()** would have normally resolved to **Base::getName()**.

However, **Base::getName()** is marked **virtual**, therefore, the compiler goes to the **most-derived implementation** of **getName()**.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}


int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(d1);

    return 0;
}
```

Virtual Functions

- The sacrifice we made: *The parent type pointer cannot see the child's members.*

- Marking our member functions **virtual** will change this behavior.

- A **virtual** function when called, resolves to the **most-derived version** of the function for the actual type of the object being referenced or pointed to.

This would have resolved to **Base::getName()** because obj is **Base&**.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}


int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(d1);

    return 0;
}
```

Because it's marked **virtual**, the compiler will go to the most derived implementation (with respect to the object being referenced by **obj**) of this method.

Note that **Derived::getName()** is overriding **Base::getName()** in this situation. Compiler is using the derived version instead of the base version.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}


int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(d1);

    return 0;
}
```

Note that **Base::getValue()** is not marked **virtual**, because there is no need for it. Even for objects of class **Derived**, I want this (inherited) method to be executed.

```cpp
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```cpp
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}


int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(d1);

    return 0;
}
```

Virtual Functions

- The sacrifice we made: *The parent type pointer cannot see the child's members.*
- Marking our member functions **virtual** will change this behavior.
- A **virtual** function when called, resolves to the **most-derived version** of the function for the actual type of the object being referenced or pointed to.
- A derived function is considered a match if it has the same signature (name, parameter types, and whether it is const) and return type as the base version of the function.
  - Such functions are called **overrides**.
  - Recommended to write **"override"** for overrides.

Note that the **overriding method** should have the **same**,

- Signature (i.e., name, parameter types, and whether it is **const**), and,
- Return type,

as the **base version** of the function

Note that even though you haven't written **virtual** *with* **Derived::getName()**, *it is implicitly assumed to be* **virtual**.

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
void whoAmI(const Base& obj)
{
    std::cout << "I am " << obj.getName()
              << " with value = " << obj.getValue()
              << '\n';
}


int main()
{
    Base b1{ 100 };
    Derived d1{ 10 };

    whoAmI(d1);

    return 0;
}
```

This **override** is optional. However, it is highly recommended to use.
Using **override** would alert you if you make any mistakes like the one shown here.

e.g. I have forgotten to add **const** in the overriding method.
*The warnings are relevant to a future topic – **virtual destructors**.*

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

    virtual std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName()          override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
tra -Wconversion -Wsign-conversion -std=c++14 .\rough.cpp
.\rough.cpp:4:7: warning: 'class Base' has virtual functions and accessible non-vi
rtual destructor [-Wnon-virtual-dtor]
 class Base
       ^~~~
.\rough.cpp:27:17: error: 'std::__cxx11::string Derived::getName()' marked 'overri
de', but does not override
     std::string getName() override
                 ^~~~~~~
.\rough.cpp:20:7: warning: base class 'class Base' has accessible non-virtual dest
ructor [-Wnon-virtual-dtor]
 class Derived: public Base
       ^~~~~~~
.\rough.cpp:20:7: warning: 'class Derived' has virtual functions and accessible no
n-virtual destructor [-Wnon-virtual-dtor]
PS C:\Users\mohammad salman\OneDrive - Habib University\Documents\Course Stuff\Fal
```

This is another error that **override** helps you with (I haven't marked the base method as **virtual**).

```
class Base
{
protected:
    int m_value {};

public:
    Base(int value)
        : m_value{ value }
    {}

        std::string getName() const
    { return "Base"; }

    int getValue() const { return m_value; }
};

class Derived: public Base
{
public:
    Derived(int value)
        : Base{ value }
    {}

    std::string getName() const override
    { return "Derived"; }

    int getValueDoubled() const { return m_value * 2; }
};
```

```
.\rough.cpp:27:17: error: 'std::__cxx11::string Derived::getName() const' marked
override', but does not override
     std::string getName() const override
                 ^~~~~~~
```

Virtual Functions

- Ensure that the function that is supposed to be overridden, is marked **virtual**.
- Ensure that functions that are meant to override should have the same return type and signature as the virtual function.
- The **override** and **virtual** tags with the functions that are meant to override are optional.

More examples to help you understand the ways **override** could help you with…

- Check the file **override.cpp**

- It has notes within comments to help you understand the problems.

(SS) **override**

Because there is no performance penalty for using the override specifier and it helps ensure you've actually overridden the function you think you have, all virtual override functions should be tagged using the override specifier. Additionally, because the override specifier implies virtual, there's no need to tag functions using the override specifier with the virtual keyword.

Best practice

Use the virtual keyword on virtual functions in a base class.

Use the override specifier (but not the virtual keyword) on override functions in derived classes. This includes virtual destructors.

*In the following reference, you can ignore the section on covariant return type. The section on the **final** specifier is also optional, but easy to understand.*

https://www.learncpp.com/cpp-tutorial/the-override-and-final-specifiers-and-covariant-return-types/

What will be the output of this program?

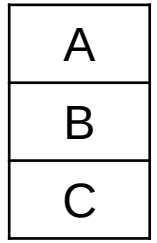
ptrA is a C

```
class A
{
public:
    virtual std::string getName() const { return "A"; }
};


class B: public A
{
public:
    std::string getName() const override { return "B"; }
};


class C: public B
{
public:
    std::string getName() const override { return "C"; }
};


class D: public C
{
public:
    std::string getName() const override { return "D"; }
};
```

```
int main()
{
    C c {};
    A* ptrA{ &c };
    std::cout << "ptrA is a " << ptrA->getName();

    return 0;
}
```

most-derived match for **getName()** *between* **A** *and* **C**

C c {};



Note that the overriding methods in the derived classes are also **virtual**.

Explanation about the *same-ish* example from learncpp

What do you think this program will output?

Let's look at how this works. First, we instantiate a C class object. rBase is an A reference, which we set to reference the A portion of the C object. Finally, we call rBase.getName(). rBase.getName() evaluates to A::getName(). However, A::getName() is virtual, so the compiler will call the most-derived match between A and C. In this case, that is C::getName(). Note that it will not call D::getName(), because our original object was a C, not a D, so only functions between A and C are considered.

As a result, our program outputs:

```
rBase is a C
```

What will be the output of this program?

ptrB is a C

```
class A
{
public:
    virtual std::string getName() const { return "A"; }
};


class B: public A
{
public:
    std::string getName() const override { return "B"; }
};


class C: public B
{
public:
    std::string getName() const override { return "C"; }
};


class D: public C
{
public:
    std::string getName() const override { return "D"; }
};
```

```
int main()
{
    C c {};
    B* ptrB{ &c };
    std::cout << "ptrB is a " << ptrB->getName();

    return 0;
}
```

Is this **virtual***?*

C c {};

A
B
C

Note that the overriding methods in the derived classes are also **virtual***.*

Runtime Polymorphism

Question
Have we seen something similar to this earlier in the course?

- What we have seen is **polymorphism** (that happens during **runtime**).

- In programming, **polymorphism** refers to the ability of an entity to have **multiple forms**.
  - "Polymorphism" literally means "many forms".

- In our example, **getName()** is taking multiple forms…
  - … depending on the **virtual function resolution**.

- This resolution happens during **runtime** (while the executable is running).

Runtime Polymorphism

- We saw **function overloading**, which is an example of **compile-time polymorphism**.
  - Compile-time implies the time when you are **compiling the code** and in the process of creating your executable file.

```
1  int add(int, int);
2  double add(double, double);
```

- Function overload resolution is done during **compile-time**.

- **Template resolution** is another example of **compile-time polymorphism**.

(SS) Another example to see if you understand things…

- Self-study the **class Animal** example under section, **“A more complex example”**.
  - Link: https://www.learncpp.com/cpp-tutorial/virtual-functions/

- You can assume **std::string_view** to be **std::string** to keep things simple to understand.

Virtual Destructors

https://www.learncpp.com/cpp-tutorial/virtual-destructors-virtual-assignment-and-overriding-virtualization/

Destruction of Child Objects

- A child/derived object is composed of multiple parts.
  - It has one part corresponding to each of its parent.
  - And it has one part that is its own.
- For child objects to be destructed correctly, all relevant destructors should be called.
  - Recall the order of destruction.
- Let’s discuss with an example.
- Here is a source file containing a similar example.
  - **virtualDtors.cpp**

Since I'm dealing with dynamic memory, I decided to make my own **Derived::~Derived()**.

```cpp
#include <iostream>
class Base
{
public:
    ~Base()
    {
        std::cout << "Calling ~Base()\n";
    }
};

class Derived: public Base
{
private:
    int* m_array {};

public:
    Derived(int length)
      : m_array{ new int[length] }
    {
    }

    ~Derived()
    {
        std::cout << "Calling ~Derived()\n";
        delete[] m_array;
    }
};
```

```cpp
int main()
{
    Derived* derived { new Derived(5) };
    Base* base { derived };

    delete base;

    return 0;
}
```

Questions

- *What happens when line 31 is executed?*
- *For line 31, could I have done this:* **Base* base{ new Derived(5) };**
- *What happens if I had skipped line 34?*
  - *Memory leak (against line 31).*

What happens when I do, **delete base;** ?

```cpp
#include <iostream>
class Base
{
public:
    ~Base()
    {
        std::cout << "Calling ~Base()\n";
    }
};

class Derived: public Base
{
private:
    int* m_array {};

public:
    Derived(int length)
      : m_array{ new int[length] }
    {
    }

    ~Derived()
    {
        std::cout << "Calling ~Derived()\n";
        delete[] m_array;
    }
};
```

```cpp
int main()
{
    Derived* derived { new Derived(5) };
    Base* base { derived };

    delete base;

    return 0;
}
```

```
Calling ~Base()
```

Whose dtor should be called?

- **base** *is a* **Base***.
- *So* **Base::~Base()** *would be called.*
- *Note that once it has executed,* **Derived::~Derived()** *doesn't get called.*
- *Not only has memory leaked, but the object also didn't destruct properly.*
  - *Top-down destruction didn't happen appropriately.*

What happens when I do, **delete base;** ?

```cpp
#include <iostream>
class Base
{
public:
    ~Base()
    {
        std::cout << "Calling ~Base()\n";
    }
};

class Derived: public Base
{
private:
    int* m_array {};

public:
    Derived(int length)
      : m_array{ new int[length] }
    {
    }

    ~Derived()
    {
        std::cout << "Calling ~Derived()\n";
        delete[] m_array;
    }
};
```

```cpp
int main()
{
    Derived* derived { new Derived(5) };
    Base* base { derived };

    delete base;

    return 0;
}
```

We actually want **Derived::~Derived()** *to be called first, since after finishing execution, this will automatically call* **Base::~Base()**.

This is where ***virtual destructors*** *come in.*

Whose dtor should be called?
base *is a* **Base***.*

So **Base::~Base()** *would be called.*

- *Note that once it has executed,* **Derived::~Derived()** *doesn't get called.*

- *Not only has memory leaked, but the object also didn't destruct properly.*

Note the modifications to the dtors.

Note that **virtual** *on line 22 is optional.*

```
1  #include <iostream>
2  class Base
3  {
4  public:
5      virtual ~Base() // note: virtual
6      {
7          std::cout << "Calling ~Base()\n";
8      }
9  };
10
11 class Derived: public Base
12 {
13 private:
14     int* m_array {};
15
16 public:
17     Derived(int length)
18       : m_array{ new int[length] }
19     {
20     }
21
22     virtual ~Derived() // note: virtual
23     {
24         std::cout << "Calling ~Derived()\n";
25         delete[] m_array;
26     }
27 };
```

```
29 int main()
30 {
31     Derived* derived { new Derived(5) };
32     Base* base { derived };
33
34     delete base;
35
36     return 0;
37 }
```

Now this program produces the following result:

```
Calling ~Derived()
Calling ~Base()
```

The call would have resolved to **Base::~Base()**, *but because it's virtual, the compiler goes to the most derived's destructor (wrt to the object being pointed to by* **base**), *which is* **Derived::~Derived()**.

Once **~Derived()** *is complete, it then automatically calls* **~Base()** *as usual.*

Virtual Destructors

- When dealing with inheritance, **always** make your destructors **virtual**.
  - Whether or not you're dealing with dynamically allocated memory.

- **(SS)** Note that there is a difference between how virtual destructors work and how other virtual methods override.
  - Can you figure it out?

- Even though the destructors in the child classes would be implicitly considered virtual, it can be a good idea to add **virtual** for the sake of readability.

Don't call virtual methods from within ctors and dtors.

What is the output of this program?

```
Creating a Derived type object.
In Base()
In Base::isBase()
In Derived()
```

```
class Base
{
private:
    int m_num;

public:
    Base() : m_num{ 0 }
    {
        std::cout << "In Base()\n";
        this->isBase();
    }

    virtual bool isBase() const
    {
        std::cout << "In Base::isBase()\n";
        return true;
    }
};
```

Any problems with these classes from what we've seen thus far?

```
class Derived: public Base
{
public:
    Derived() : Base{}
    {
        std::cout << "In Derived()\n";
    }

    bool isBase() const override
    {
        std::cout << "In Derived::isBase()\n";
        return false;
    }
};
```

Any problems with **main()***?*

```
int main()
{
    std::cout << "Creating a Derived type object.\n";
    Base* pBase{ new Derived };

    return 0;
}
```